

Compile-time vs Runtime Polymorphism

The big idea:

Compile-time polymorphism: The compiler decides which function to call **before the program runs**.

Runtime polymorphism: The decision is made **while the program is running**.

Both are forms of **polymorphism**.

1. The big picture

Compile-timeRuntimeAlso calledStatic / early bindingDynamic / late bindingDecision madeDuring compilationDuring executionMain mechanismsFunction overloading, operator overloading, templatesVirtual functionsInheritance required?NoUsually yesvirtual required?NoYesMain idea"Which version matches?"Which object's implementation?"

2. Function Overloading → Compile-time

Same function name, **different parameters**.

```
class Calculator {
public:
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
};
```

Now:

```
Calculator c;

c.add(2, 3);           // int version
c.add(2.5, 3.5);       // double version
```

The compiler determines the correct function based on the **arguments**.

So this is:

Function overloading = compile-time polymorphism

Interview question

Q: Can function overloading differ only by return type?

No.

```
int add(int a, int b);
double add(int a, int b); // ✗
```

Why?

Because the call:

```
add(2, 3);
```

doesn't provide enough information for the compiler to distinguish them.

3. Operator Overloading → Compile-time

C++ lets us define how operators behave for our classes.

```
class Complex {
public:
    int real, imag;

    Complex(int r, int i) : real(r), imag(i) {}

    Complex operator+(const Complex& other) {
        return Complex(real + other.real,
                        imag + other.imag);
    }
};
```

Now:

```
Complex a(2, 3);
Complex b(4, 5);

Complex c = a + b;
```

The compiler essentially resolves:

```
a.operator+(b);
```

So:

Operator overloading = compile-time polymorphism

Important interview point

Operator overloading does **not** mean you can create completely new operators.

You can overload existing operators such as:

```
+
-
*
/
==
<
[]
()
<<
```

but you cannot invent something like:

```
**    // ✗
```

4. Templates → Compile-time Polymorphism

Templates allow generic code.

```
template <typename T>
T maximum(T a, T b) {
    return a > b ? a : b;
}
```

Now:

```
maximum(10, 20);    // T = int
maximum(2.5, 3.5);  // T = double
```

The compiler generates the appropriate version.

Conceptually:

```
maximum<int>()
maximum<double>()
```

So:

Templates = compile-time polymorphism

This is sometimes called **parametric polymorphism**.

Interview question

Q: Why are templates considered compile-time polymorphism?

Because template instantiation happens during compilation, and the compiler generates code specialized for the required types.

5. Runtime Polymorphism → Virtual Functions

This is the **most important part** for interviews.

Consider:

```
class Animal {
public:
    virtual void speak() {
        cout << "Animal speaks\n";
    }
};

class Dog : public Animal {
public:
    void speak() override {
        cout << "Dog barks\n";
    }
};

class Cat : public Animal {
public:
    void speak() override {
        cout << "Cat meows\n";
    }
};
```

Now:

```
Animal* a = new Dog();

a->speak();
```

Output:

```
Dog barks
```

Why?

Because `speak()` is **virtual**.

At runtime, C++ determines:

```
a
↓
actually points to
↓
Dog object
↓
call Dog::speak()
```

This is **runtime polymorphism**.

6. Why **virtual** matters

Remove **virtual**:

```
class Animal {
public:
    void speak() {
        cout << "Animal speaks\n";
    }
};
```

Then:

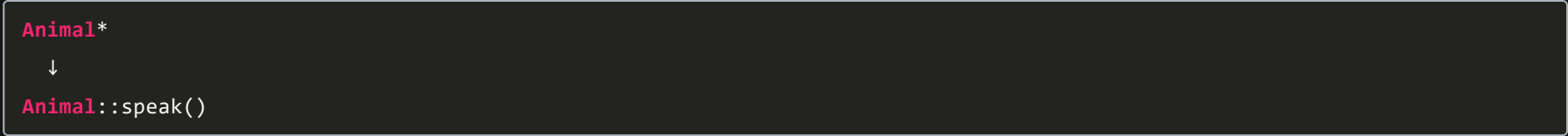
```
Animal* a = new Dog();

a->speak();
```

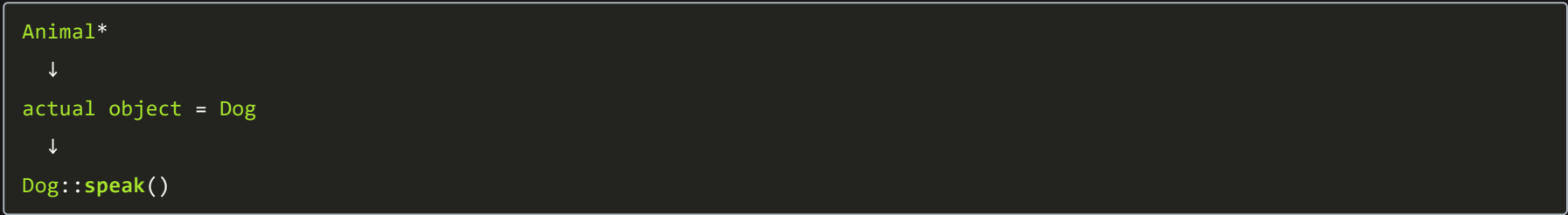
Output:

```
Animal speaks
```

Because without **virtual**, the function call is resolved based on the **pointer/reference type**:



With `virtual`:



That's the heart of runtime polymorphism.

7. Static vs Dynamic Binding

This terminology is very important in interviews.

Without virtual

```
Animal* a = new Dog();
a->speak();
```

The compiler binds:

```
Animal* → Animal::speak()
```

This is **static binding** / **early binding**.

With virtual

```
Animal* a = new Dog();
a->speak();
```

The runtime dispatches:

```
actual object → Dog::speak()
```

This is **dynamic binding** / **late binding**.

8. How does C++ implement runtime polymorphism?

This is a classic interview follow-up.

Most C++ implementations use:

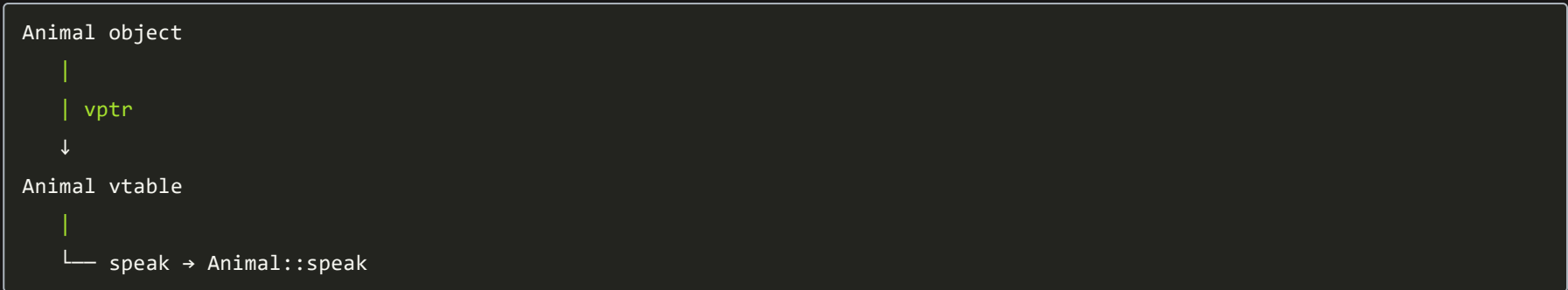
vtable + vptr

Suppose:

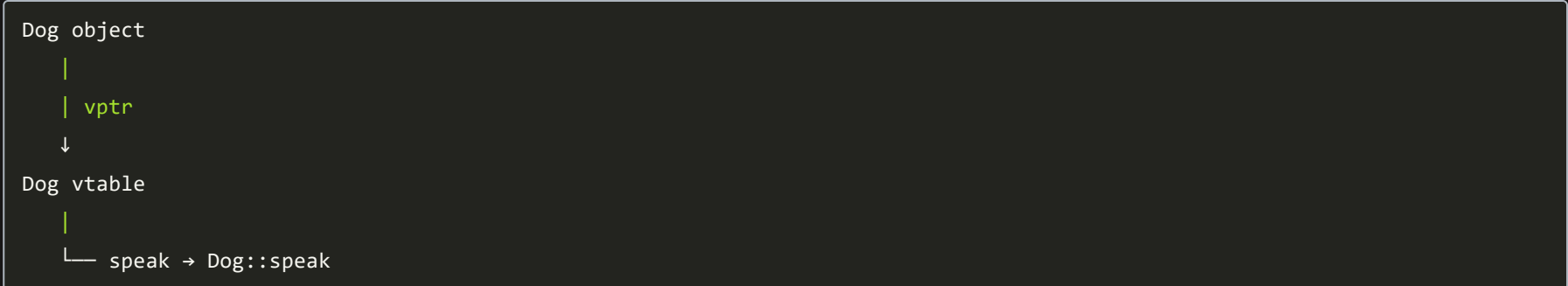
```
class Animal {
public:
    virtual void speak();
};

class Dog : public Animal {
public:
    void speak() override;
};
```

Conceptually:



Dog:

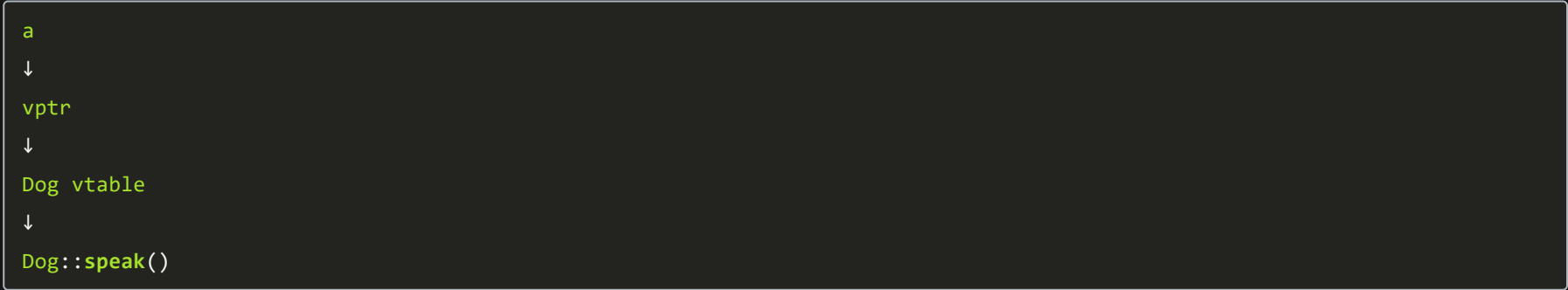


So:

```
Animal* a = new Dog();

a->speak();
```

roughly works through:



Interview warning

Don't say:

```
"C++ standard guarantees vtable and vptr."
```

It doesn't.

Better answer:

```
"Runtime polymorphism is typically implemented using a virtual table (vtable) and a hidden virtual-table pointer (vptr), although the C++ standard doesn't mandate this particular implementation."
```

That's a **strong interview answer**.

9. Why do we need a base pointer/reference?

This is critical.

Runtime polymorphism typically looks like:

```
Animal* a = new Dog();
```

or:

```
Animal& a = dog;
```

because the **base type provides a common interface**, while the actual object can be derived.

For example:

```
vector<Animal*> animals;

animals.push_back(new Dog());
animals.push_back(new Cat());

for (Animal* animal : animals) {
    animal->speak();
}
```

Output:

```
Dog barks
Cat meows
```

Same interface:

```
animal->speak();
```

Different behavior depending on the actual object.

That's polymorphism.

10. Very important: Overloading vs Overriding

Interviewers LOVE this.

Overloading

Same class:

```
class A {
public:
    void foo(int);
    void foo(double);
};
```

Different parameter list.

➡ **Compile-time**

Overriding

Base + derived:

```
class Animal {
public:
    virtual void speak();
};

class Dog : public Animal {
public:
    void speak() override;
};
```

Same function signature, derived class provides new implementation.

➡ **Runtime**, when called virtually through base pointer/reference.

Remember:

Overloading = same name, different parameters

Overriding = derived class replaces virtual base implementation

11. A common interview trap

They may ask:

"Is function overriding always runtime polymorphism?"

Not necessarily.

You need **virtual dispatch** for runtime polymorphism through a base pointer/reference.

Example:

```
class Base {
public:
    void show() {
        cout << "Base";
    }
};

class Derived : public Base {
public:
    void show() {
        cout << "Derived";
    }
};
```

Now:

```
Base* p = new Derived();
p->show();
```

Output:

Base

This is **not runtime polymorphism**.

Because `show()` isn't virtual.

12. `override` keyword

You should almost always use:

```
void speak() override
```

instead of simply:

```
void speak()
```

Why?

The compiler verifies that you're actually overriding a virtual function.

Example:

```
class Base {
public:
    virtual void show(int);
};

class Derived : public Base {
public:
    void show() override; // ✗ compile error
};
```

The signatures don't match.

Without `override`, this mistake could silently create a different function.

13. Compile-time vs Runtime example

Think of a restaurant.

Compile-time

You say:

```
"Give me a pizza."
```

The system knows from your request exactly which overloaded function applies.

```
order(int)
order(string)
```

Compiler chooses based on arguments.

Runtime

You say:

```
"Make this Animal speak."
```

The actual animal could be:

```
Dog
Cat
Cow
```

You don't know until runtime.

```
Animal* animal;
animal->speak();
```

The actual object determines the behavior.

14. Performance difference

Compile-time polymorphism generally has **less runtime overhead** because the function is resolved during compilation.

Runtime polymorphism can involve:

```
indirection through vtable
```

so there can be a small dispatch overhead.

But don't say:

```
"Virtual functions are always slow."
```

That's too strong.

Modern compilers can sometimes **devirtualize** calls when they can determine the concrete type.

Interview answer:

"Compile-time polymorphism generally avoids dynamic dispatch overhead, while runtime polymorphism may incur a small virtual dispatch cost."

15. The most important examples to memorize

Compile-time

```
// Function overloading
void print(int);
void print(string);

// Operator overloading
Complex operator+(const Complex&);

// Templates
template<typename T>
T max(T a, T b);
```

Runtime

```
class Base {
public:
    virtual void show();
};

class Derived : public Base {
public:
    void show() override;
};

Base* p = new Derived();
p->show(); // Derived::show()
```

Interview Questions You Should Be Able to Answer

Basic

1. What is polymorphism?

Ability of the same interface/name to exhibit different behavior depending on context.

2. What are the two types of polymorphism in C++?

```
Compile-time polymorphism
├─ Function overloading
├─ Operator overloading
└─ Templates

Runtime polymorphism
└─ Virtual functions
```

3. What is function overloading?

Same function name with different parameter lists. Resolved at compile time.

4. What is function overriding?

Derived class provides its own implementation of a virtual function inherited from the base class.

5. Difference between overloading and overriding?

Overloading	Overriding	Same name	Same function signature	Usually same class	Base + derived	Different parameters	Same parameters
Compile-time	Runtime with virtual dispatch						

Medium

6. Why is `virtual` required for runtime polymorphism?

Because it enables dynamic dispatch, allowing the call through a base pointer/reference to resolve to the derived class implementation.

7. What happens if you don't use `virtual`?

The call is statically bound based on the pointer/reference's compile-time type.

8. What is a vtable?

A compiler-implementation mechanism commonly used to store addresses of virtual functions for a class.

9. What is a vptr?

A hidden pointer typically stored in polymorphic objects that points to the appropriate vtable.

10. Why do we use `override`?

To ask the compiler to verify that a derived function actually overrides a base virtual function.

Tricky

11. Can constructors be virtual?

No. Virtual dispatch requires an already-existing object's dynamic type, but during construction the object is still being constructed.

12. Can destructors be virtual?

Yes, and a base destructor **should usually be virtual** when deleting derived objects through a base pointer.

```
class Base {  
public:  
    virtual ~Base() {}  
};
```

13. Can static functions be virtual?

No. Static functions don't belong to a particular object, so there is no dynamic dispatch based on an object's runtime type.

14. Can a private function be overridden?

A derived class cannot directly access a private base function, and a private base member isn't overridden in the usual polymorphic sense.

One interview question you should definitely practice

What is the output?

```
class Base {
public:
    virtual void show() {
        cout << "Base";
    }
};

class Derived : public Base {
public:
    void show() override {
        cout << "Derived";
    }
};

int main() {
    Base b;
    Derived d;

    Base* p1 = &b;
    Base* p2 = &d;

    p1->show();
    p2->show();
}
```

Answer:

BaseDerived

Reason:

p1 → actual object Base → Base::show()
p2 → actual object Derived → Derived::show()

That's runtime polymorphism.

🧠 Final Mental Model

Remember this:



One-liner for interviews

Compile-time polymorphism is resolved by the compiler using mechanisms such as overloading, operator overloading, and templates, whereas runtime polymorphism uses virtual functions and dynamic dispatch to select the implementation based on the object's actual runtime type.

That distinction is worth memorizing almost word-for-word.

